Initiation au Développement : Algorithmique Fondamentale, Variables et Structures de Contrôle

Thème principal : Algorithmique et Programmation de Base

Synthèse exécutive : Ce cours pose les fondations absolues de la pensée algorithmique et du développement logiciel pour les étudiants de BUT Informatique. Il explique comment un ordinateur manipule la mémoire vive (RAM) à travers le concept de variables modélisées par un quadruplet (Nom, Type, Valeur, Adresse) et l'affectation destructive. Il aborde également l'aiguillage du flux d'exécution séquentiel grâce aux structures conditionnelles et aux tables de vérité, ainsi que l'automatisation des traitements répétitifs via les boucles bornées (Pour) et non bornées (TantQue).

Mémorisation et Manipulation des Données : Les Variables en RAM

En informatique, une variable se distingue radicalement d'une variable mathématique statique. Il s'agit d'une case mémoire physique concrète située dans la RAM. Lors d'une instruction d'affectation (ex: x ← x + 1), le processeur évalue l'expression de droite puis inscrit le résultat dans la case de gauche en écrasant l'ancienne valeur. Oublier d'initialiser une variable conduit à lire des bits résiduels, créant un comportement erroné.

- Variable : Emplacement mémoire étiqueté réservé dans la RAM pour stocker une valeur modifiable.
- Quadruplet : Nom, Type, Valeur et Adresse physique de la variable.
- Affectation : Opération destructive d'écriture évaluant la droite et écrasant la gauche.
- Valeur poubelle : Donnée résiduelle lue en RAM lorsqu'une variable n'a pas été initialisée.

Cas concret enseignant : *La métaphore de l'armoire d'archivage à tiroirs : Déclarer une variable revient à coller une étiquette sur un tiroir (ex: 'solde_compte') et à dimensionner sa profondeur selon son type ('Entier'). L'affectation 'solde_compte ← 150' consiste à ouvrir le tiroir, jeter l'ancien papier et y déposer la nouvelle valeur.*

Prise de Décision : Les Structures Conditionnelles (Si / Sinon)

Par défaut, un processeur exécute les instructions de manière séquentielle. Les structures conditionnelles permettent d'évaluer un prédicat booléen : si la condition est vraie, le bloc 'Alors' est exécuté et le bloc 'Sinon' est ignoré (et inversement).

- Structure d'Alternative : Aiguillage logique détournant le flux d'exécution séquentiel.
- Expression Booléenne : Prédicat évalué strictement à VRAI ou FAUX.
- Opérateurs Logiques : ET, OU, NON régis par les tables de vérité.

Cas concret enseignant : *Le cas réel du Distributeur Automatique de Billets (DAB) : Avant d'éjecter l'argent, l'automate évalue 'SI (solde_client >= montant_demandé) ET (billets_disponibles >= montant_demandé)'. Si l'expression est vraie, les billets sont délivrés ; sinon, une erreur est affichée et la carte est restituée.*

Répétition et Automatisation : Les Structures Itératives (Boucles)

Les boucles exploitent la vitesse du processeur pour répéter des instructions. La boucle 'Pour' gère nativement un compteur, tandis que la boucle 'TantQue' vérifie une condition avant chaque passage. La règle d'or du 'TantQue' est de modifier la variable de contrôle dans le corps de la boucle pour éviter le blocage total.

- Boucle Pour (FOR) : Structure itérative bornée pour un nombre de tours connu.
- Boucle TantQue (WHILE) : Structure itérative non bornée évaluée dynamiquement.
- Boucle infinie : Blocage du processeur dû à l'absence de modification de la variable de contrôle.

Cas concret enseignant : *La métaphore de l'entraîneur d'athlétisme : 'Fais 10 tours de piste' correspond à une boucle POUR (le nombre 10 est fixé au départ). 'Cours tant qu'il ne pleut pas' correspond à une boucle TANT QUE (la fin dépend d'un paramètre météo réévalué à chaque tour).*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi l'instruction 'x = x + 1' (ou x ← x + 1) est-elle valide en programmation alors qu'elle est mathématiquement impossible ?
Réponse du professeur : Parce qu'en informatique, le symbole '←' n'est pas une équation mathématique d'égalité mais un ordre d'affectation : on évalue d'abord l'expression de droite (x + 1) en utilisant la valeur actuelle stockée en RAM, puis on range le résultat dans la variable x, écrasant ainsi l'ancienne valeur.

Question étudiant : Que se passe-t-il concrètement si j'oublie de modifier la variable de contrôle à l'intérieur du corps d'une boucle TantQue ?
Réponse du professeur : Le programme entre dans une boucle infinie : la condition reste éternellement vraie, l'ordinateur répète indéfiniment les mêmes instructions sans jamais sortir de la boucle, ce qui gèle l'application et sature le processeur à 100%.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un algorithme en pseudo-code qui demande à l'utilisateur de saisir un nombre entier positif N, puis calcule et affiche la somme accumulée de tous les entiers compris entre 1 et N inclus (Somme = 1 + 2 + ... + N).

Démarche de résolution :
Étape 1 : Déclarer les variables N (Entier), i (Entier, compteur) et somme (Entier, accumulateur). Étape 2 : Demander et lire la valeur de N. Étape 3 : Initialiser strictement l'accumulateur : somme ← 0 (pour purger toute valeur poubelle en RAM). Étape 4 : Mettre en place la boucle Pour : Pour i allant de 1 à N par pas de 1 Faire somme ← somme + i FinPour. Étape 5 : Afficher le résultat final contenu dans la variable somme.

Résultat : Pour une saisie N = 5 : i=1 -> somme=1 ; i=2 -> somme=3 ; i=3 -> somme=6 ; i=4 -> somme=10 ; i=5 -> somme=15. Le programme affiche '15'.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale entre une variable informatique et une variable mathématique ?

R : Une variable informatique est un conteneur physique modifiable situé dans la RAM, associé à un type et soumis à une affectation destructive, tandis qu'une variable mathématique est une inconnue statique dans une équation.

Q : Que valent les variables A et B après l'exécution de la séquence : A ← 5 ; B ← 3 ; A ← A + B ; B ← A - B ?

R : A vaut 8 (5 + 3), et B vaut 5 (8 - 3). C'est l'algorithme classique d'échange de valeurs sans variable temporaire.

Q : Pourquoi l'absence d'initialisation d'une variable d'accumulation (ex: somme ← 0) fait-elle perdre des points en examen ?

R : Parce que l'absence d'initialisation force le processeur à lire une 'valeur poubelle' résiduelle en RAM, faussant totalement le calcul dès la première itération.